Exercise 1)

The set of all possible five-card poker hands

a) How many atomic events are there in the joint probability distribution?

$$\text{Atomic events are } \binom{52}{5} = \frac{52!}{5!47!} = \underline{\underline{2\,598\,960}}$$

b) What is the probability of each atomic event?

$$P(\text{each event}) = \frac{1}{2\,598\,960} = \underline{\underline{3{,}85 \cdot 10^{-7}}}$$

c) What is the probability for a royal straight flush?

$$P(\text{royal flush}) = \frac{4}{2\,598\,960} = \underline{\underline{2{,}401 \cdot 10^{-4}}}$$

Exercise 2)

Deciding to put probability theory to good use, we encounter a slot machine with three independent wheels, each producing one of the four symbols BAR, BELL, LEMON, or CHERRY with equal probability. The slot machine has the following payout scheme for a bet of 1 coin (where "?" denotes that we don't care what comes up for that wheel):

BAR/BAR/BAR pays 20 coins
BELL/BELL/BELL pays 15 coins
LEMON/LEMON/LEMON pays 5 coins
CHERRY/CHERRY/CHERRY pays 3 coins
CHERRY/CHERRY/? pays 2 coins
CHERRY/?/? pays 1 coin

The **payouts do not stack**. For instance, if you get 3 cherries, you get 3 coins. You **do not** get 6 coins, because of 3 coins for 3 cherries, 2 coins for 2 cherries and 1 coin for 1 cherry. The wheel order matters. Cherries need to be on specific wheels as outlined in the list. Some examples: CHERRY/CHERRY/BAR pays 2, CHERRY/BAR/CHERRY **pays 1**, CHERRY/BAR/BAR pays 1 and BAR/CHERRY/BAR **pays 0**.

a. Compute the expected "payback" percentage of the machine. In other words, for each coin played, what is the expected coin return?

b. Compute the probability that playing the slot machine once will result in a win.

c. Estimate the mean and median number of plays you can expect to make until you go broke, if you start with 10 coins. Run a simulation in Python to estimate this. Add your results to your PDF report.

a) For one coin

$P(\text{one CHERRY}) = \frac{1}{4}$, $P(\text{two CHERRY}) = \frac{1}{16}$ $P(\text{three CHERRY}) = \frac{1}{64}$

two CHERRY + x → 3 possibilities give 2 coins
one CHERRY + x + y → 12 possibilities give 1 coin

$$E[X] = \frac{1}{64}(20+15+5+3) + \frac{3}{64}\cdot 2 + \frac{12}{64} = \frac{61}{64} = 0{,}9531 = \underline{95{,}31\%}$$

b) We have 64 different solutions which 19 of them are considered a win

$$P(\text{win}) = \frac{19}{64} = 0{,}296875 \approx \underline{29{,}69\%}$$

c)

```
import numpy as np
import random
```
✓ 0.0s

```
symbols = ["CHERRY", "LEMON", "BELL", "BAR"]

#Function to spinn the slot machine
def spin_reels():
    return [random.choice(symbols) for _ in range(3)]
```
✓ 0.0s

```
#Function to calculate payback
def payout(reels):
    if reels[0] == "BAR" and reels[1] == "BAR" and reels[2] == "BAR":
        return 20
    elif reels[0] == "BELL" and reels[1] == "BELL" and reels[2] == "BELL":
        return 15
    elif reels[0] == "LEMON" and reels[1] == "LEMON" and reels[2] == "LEMON":
        return 5
    elif reels[0] == "CHERRY" and reels[1] == "CHERRY" and reels[2] == "CHERRY":
        return 3
    elif reels[0] == "CHERRY" and reels[1] == "CHERRY":
        return 2
    elif reels[0] == "CHERRY":
        return 1
    else:
        return 0
```
✓ 0.0s

```
def play(start_cash):
    cash = start_cash
    rounds = 0

    while cash>0:
        reels = spin_reels()
        win = payout(reels)
        cash +=  - 1 + win
        rounds += 1

    return rounds


#Number of simulations
N = 10000

all_rounds = []

for _ in range(N):
    all_rounds.append(play(10))

mean_rounds = np.mean(all_rounds)
median_rounds = np.median(all_rounds)

print("Mean number of rounds", mean_rounds)
print("Median number of rounds", median_rounds)
```
✓ 1.9s

```
Mean number of rounds 212.3904
Medina number of rounds 21.0
```

Exercise 3

Part 1

Peter is interested in knowing the possibility that at least two people from a group of N people have a birthday on the same day. Your task is to find out what N has to be for this event to occur with at least 50% chance. We will disregard the existence of leap years and assume there are 365 days in a year that are equally likely to be the birthday of a randomly selected person.

a. Create a function that takes N and computes the probability of the event via simulation.

b. Use the function created in the previous task to compute the probability of the event given N in the interval $[10, 50]$. In this interval, what is the proportion of N where the event happens with the least 50% chance? What is the smallest N where the probability of the event occurring is at least 50%?

Part 2

Peter wants to form a group where every day of the year is a birthday (i.e., for every day of the year, there must be at least one person from the group who has a birthday). He starts with an empty group, and then proceeds with the following loop:

1. Add a random person to the group.
2. Check whether all days of the year are covered.
3. Go back to step 1 if not all days of the year have at least one birthday person from the group.

a. How large a group should Peter expect to form? Make the same assumption about leap years as in Part 1.

Part 1)

```
import random
```
✓ 0.0s

```
def random_day():
    return random.randint(1, 365)
```
✓ 0.0s

```
N = 23
count = 0
trials = 20000



def birthday(N):
    birthdays = []
    for _ in range(N):
        birthdays.append(random_day())
        if len(set(birthdays)) < len(birthdays):
            return True
    return False



for _ in range(trials):
    if birthday(N):
        count += 1

p_hat = count / trials
print("P(shared birthday) =", p_hat)
```
✓ 0.1s

```
P(shared birthday) = 0.5065
```

```
N_values = list(range(10, 51))


for N in N_values:
    count = 0
    for _ in range(trials):
        if birthday(N):
            count += 1
    p_hat = count / trials
    print(N, p_hat)
```
✓ 10.1s

```
10 0.11715
11 0.1416
12 0.1637
13 0.19625
14 0.2239
15 0.25285
16 0.2886
17 0.3179
18 0.34565
19 0.3851
20 0.41015
21 0.4503
22 0.47885
23 0.50845
24 0.5391
25 0.5716
26 0.59625
27 0.62155
28 0.65275
29 0.6849
30 0.702
31 0.7351
32 0.75845
33 0.77405
34 0.79005
```

a) Needed 23 persons to get P ≥ 0,5

b) After N ≥ 23 it is a 50% chance for birthday on the same day

Part 2

```
#Find number of people needed to cover every birthday
def every_birthday():
    birthdays = []
    while len(set(birthdays)) < 365:
            birthdays.append(random_day())

    return len(birthdays)


print("Number of people needed for every birthday:",every_birthday())
```
✓ 0.0s

```
Number of people needed for every birthday: 2631
```

Expect a group of 2631 persons.